irrelevant). But it also depends at what price you get the saving: if it makes your code terribly more fragile, more complicated and harder to maintain, or just requires weeks of development time, you will have to weight PROs and CONs before going down this path.
Luckily for disjoint sets, this is not the case, and path compression is easy to implement, while it gives a big gain.

Let's see how we could have this further improvement, before delving into code.

5.6.1 Path Compression

As hinted in the previous section, we can do even better than just having balanced trees and logarithmic-time methods.

To improve our result further, we can use a heuristic called *path compression*. The idea is even simpler: for each node in the trees, instead of storing a link to its parent, we can store one to the root of the tree. After all, we don't really need to keep track of the history of the merges we performed, we just need to know at current time what's the root for an element’s partition - and find that out as quickly as we can.